

Artificial Intelligence & Machine Learning Fundamentals

A comprehensive study guide covering core concepts, algorithms, and practice

Chapter 1: What Is Machine Learning?

Machine Learning (ML) is a branch of Artificial Intelligence that enables computers to learn from data and improve their performance on tasks without being explicitly programmed for each scenario. Instead of writing rules by hand, we feed examples to an algorithm and let it discover patterns automatically. This paradigm shift has powered breakthroughs in image recognition, natural language processing, recommendation systems, and autonomous vehicles.

There are three primary learning paradigms:

- **Supervised Learning** — The algorithm is trained on labelled data (input-output pairs). Given a house's size and location, predict its price. The model minimises the error between its predictions and the true labels.
- **Unsupervised Learning** — No labels are provided. The algorithm finds hidden structure in data — grouping similar customers together (clustering) or compressing data while retaining its most important features (dimensionality reduction).
- **Reinforcement Learning** — An agent interacts with an environment, receives rewards or penalties, and learns a policy that maximises cumulative reward. This is how AlphaGo mastered the game of Go.

1.1 The ML Workflow

Every ML project follows a broadly consistent pipeline regardless of domain:

1. Problem Definition	Clearly state what you want to predict or optimise. A vague problem statement is the #1 cause of failed projects.
2. Data Collection	Gather raw data from databases, APIs, sensors, or web scraping. Data quality trumps model complexity.
3. Exploratory Analysis	Visualise distributions, detect outliers, understand correlations before modelling.
4. Feature Engineering	Transform raw inputs into representations that algorithms can exploit — encode categories, scale numerics, create interaction terms.

5. Model Selection & Training	Choose an algorithm family, split data into train/validation/test sets, fit the model.
6. Evaluation	Measure performance with appropriate metrics (accuracy, F1, RMSE, AUC-ROC).
7. Deployment & Monitoring	Serve predictions via an API; monitor for data drift and model degradation over time.

Chapter 2: Linear Regression — Learning from Numbers

Linear regression is the simplest and most interpretable supervised learning algorithm. It models the relationship between a continuous output y and one or more input features X as a straight line (or hyperplane in higher dimensions):

$$y = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n + \epsilon$$

where w_0 is the bias (intercept), $w_1 \dots w_n$ are learned weights, and ϵ is the irreducible noise term.

The model is trained by minimising the Mean Squared Error (MSE) loss function using Gradient Descent — an iterative optimisation algorithm that nudges the weights in the direction that reduces error:

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Generate synthetic house price data
np.random.seed(42)
X = np.random.rand(200, 1) * 200 # size in sq meters
y = 1500 * X.ravel() + np.random.randn(200) * 50000 + 100000 # price in USD

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print(f"R-squared : {r2_score(y_test, y_pred):.4f}")
print(f"RMSE      : {np.sqrt(mean_squared_error(y_test, y_pred)):.0f} USD")
print(f"Coefficient: {model.coef_[0]:.2f} Intercept: {model.intercept_:,.0f}")
```

R-squared (R^2) measures the proportion of variance in y explained by the model. A value of 1.0 is a perfect fit; 0 means the model does no better than predicting the mean. RMSE gives error in the original units, making it easy to interpret.

Chapter 3: Classification & Logistic Regression

When the output is a discrete class label (spam/not-spam, benign/malignant, cat/dog) we need classification algorithms. Logistic Regression adapts linear regression by passing the output through a sigmoid function that squashes any real number into the range (0, 1), interpretable as a probability:

$P(y=1 | X) = 1 / (1 + e^{(-z)})$ where $z = w \cdot X + b$ The decision boundary is typically set at 0.5: predict class 1 if $P > 0.5$.

Key evaluation metrics for classifiers:

Accuracy: Correct predictions / Total predictions. Misleading for imbalanced datasets.

Precision: $TP / (TP + FP)$. Of all predicted positives, how many were actually positive?

Recall: $TP / (TP + FN)$. Of all true positives, how many did we catch?

F1-Score: Harmonic mean of precision and recall. Balances both concerns.

AUC-ROC: Area under the Receiver Operating Characteristic curve. Threshold-independent measure of separability.

3.1 Decision Trees & Random Forests

A Decision Tree learns a hierarchy of if-else rules by recursively splitting the data on the feature that maximises information gain (or minimises Gini impurity). Trees are highly interpretable but prone to overfitting. Random Forests overcome this by training hundreds of trees on random subsets of data and features, then averaging their predictions — a technique called bagging:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.metrics import classification_report

data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42)

rf = RandomForestClassifier(n_estimators=100, max_depth=6, random_state=42)
rf.fit(X_train, y_train)
print(classification_report(y_test, rf.predict(X_test)))

# Feature importance
importances = rf.feature_importances_
top5 = sorted(zip(data.feature_names, importances), key=lambda x: -x[1])[5]
for feat, imp in top5:
    print(f" {feat:30s}: {imp:.4f}")
```

Chapter 4: Neural Networks & Deep Learning

Artificial Neural Networks (ANNs) are loosely inspired by the brain. They consist of layers of interconnected nodes (neurons). Each neuron computes a weighted sum of its inputs and passes it through a non-linear activation function. The key insight is that stacking multiple layers allows the network to learn hierarchical representations — edges in layer 1, shapes in layer 2, objects in layer 3.

Common activation functions:

ReLU (Rectified Linear Unit): $f(x) = \max(0, x)$. Fast, sparse, default choice for hidden layers.

Sigmoid: $f(x) = 1/(1+e^{-x})$. Outputs (0,1); used in binary output neurons.

Softmax: Converts a vector of logits into a probability distribution. Used in multi-class output layers.

Tanh: Outputs (-1,1). Zero-centred, preferred over sigmoid in hidden layers in some architectures.

4.1 Building a Neural Network with Keras

```
import tensorflow as tf
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Dense(128, activation='relu', input_shape=(30,)),
    layers.Dropout(0.3),          # randomly zero 30% of neurons to prevent overfitting
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid') # binary classification output
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy', tf.keras.metrics.AUC(name='auc')]
)

history = model.fit(
    X_train, y_train,
    epochs=30, batch_size=32,
    validation_split=0.15,
    callbacks=[tf.keras.callbacks.EarlyStopping(patience=5, restore_best_weights=True)]
)
```

Backpropagation calculates the gradient of the loss with respect to every weight using the chain rule, enabling the optimiser (Adam, SGD) to update weights in the direction that reduces loss. Dropout regularisation randomly disables neurons during training, preventing co-adaptation and reducing overfitting.

Chapter 5: Natural Language Processing

NLP enables machines to understand, generate, and reason about human language. Modern NLP is dominated by the Transformer architecture, which replaces recurrence with self-attention — allowing every token to directly attend to every other token in a sequence regardless of distance.

Core NLP pipeline steps:

- **Tokenisation** — Split text into tokens (words, subwords, or characters).
- **Embedding** — Map tokens to dense vectors. Word2Vec, GloVe, or contextual embeddings from BERT.
- **Encoding** — A stack of Transformer encoder layers produces context-aware representations.
- **Task Head** — A lightweight layer on top performs classification, generation, QA, etc.

```
from transformers import pipeline

# Zero-shot text classification – no fine-tuning needed
classifier = pipeline("zero-shot-classification",
                      model="facebook/bart-large-mnli")

text = "The stock market fell sharply after the Fed raised interest rates."
labels = ["finance", "sports", "politics", "technology", "health"]

result = classifier(text, candidate_labels=labels)
for label, score in zip(result["labels"], result["scores"]):
    print(f"{label:15s}: {score:.3f}")
```

Key Insight: Large Language Models (LLMs) like GPT-4 are trained to predict the next token on trillions of words. Through this simple objective, they implicitly learn grammar, facts, reasoning, and even coding — an emergent property of scale.